

UNIVERSITY OF TORONTO
Department of Computer Science

CSC263

Data Structures and Analysis

Lecture Notes
Lectures 1 & 2

Fall 2026
Instructor: Sam Toueg

Cheryl Shi

Contents

Lecture 1: Worst Case Time Complexity	1
1 Steps and Input Size	1
2 Worst Case Time Complexity	1
3 Proving Bounds on $T(n)$	1
4 Two Issues $\Rightarrow$ Asymptotic Notation	2
5 Big O (Upper Bound)	2
6 Big Ω (Lower Bound)	2
7 Big Θ (Tight Bound)	3
8 Worked Example: Bubble Sort is $\Theta(n^2)$	3
 Lecture 2: Priority Queues and Max-Heaps	 5
9 ADT vs. Data Structure	5
10 The Priority Queue ADT	5
11 Naive Implementations: Why We Need Something Better	5
12 Complete Binary Tree (CBT)	5
12.1 Height	6
13 Max-Heap	6
14 Array Representation	7
15 The Three Operations	8
15.1 Max(A)	8
15.2 Insert(A, x): percolate up	8
15.3 Extract_Max(A): drip down	9
16 Complexity Analysis	10
17 HeapSort	11
18 Past Exam: k -Set ADT	11
Quick Reference: Lecture 2	12
 Lecture 3: Min Binomial Heaps	 13
19 Mergeable Priority Queue ADT	13
20 Binomial Trees	13
20.1 Recursive Definition of B_k	13
20.2 Children Sequence of B_k	14
20.3 Properties of B_k	14
20.4 Why “Binomial”?	15
21 Binomial Forest F_n	15
21.1 Connection to Binary Representation	15
21.2 What is $\alpha(n)$?	16
21.3 Properties of F_n	16
22 Min Binomial Heap: The Full Definition	16
23 Building a Binomial Heap: Worked Example	17
24 Memory Representation	17
24.1 Three Pointers Per Node	17

Lecture 1: Worst Case Time Complexity

1 Steps and Input Size

We count **basic operations**, each taking constant time:

Arithmetic	+, −, *, /
Comparisons	>, <, =
Data moves	load, store, copy

Input size n depends on the problem: number of array elements (sorting), number of vertices + edges (graphs), or number of bits (primality).

2 Worst Case Time Complexity

Worst case time complexity $T(n)$

Let A be an algorithm and $t(x)$ = steps A takes on input x .

$$T(n) = \max\{t(x) \mid x \text{ is an input of size } n\}$$

$T(n)$ = the **maximum** steps over *all* inputs of size n .

★ Seen on a past exam (Midterm, Fall 2014, Q1a)

```
FindLast(A, x):
    i ← A.length − 1
    while i ≥ 0 and A[i] ≠ x:
        i ← i − 1
    return i
```

Q. What is the worst case complexity of FindLast? (Count array accesses only.)

A. $T(n) = n$.

Worst case: $x \notin A$, so we scan every element exactly once $\Rightarrow n$ accesses. Cannot exceed n , so $T(n) = n$.

3 Proving Bounds on $T(n)$

The key: unpack what max means.

The max trick

$$\max(S) \leq c \iff \forall e \in S : e \leq c$$

$$\max(S) \geq c \iff \exists e \in S : e \geq c$$

Apply to $T(n) = \max\{t(x)\}$:

$T(n) \leq 7n^3 \iff$ for **every** input of size n , A takes **at most** $7n^3$ steps

$T(n) \geq 3n^2 \iff$ for **some** input of size n , A takes **at least** $3n^2$ steps

Example: proving $T(n) \geq 3n$ for FindLast

We need **some** input of size n with $t(x) \geq 3n$. Hmm, FindLast does at most n accesses, so $T(n) \geq 3n$ is **false**. But $T(n) \geq n$ is true: choose $x \notin A$, then $t(x) = n \geq n$. ✓

4 Two Issues $\Rightarrow$ Asymptotic Notation

$T(n) \leq 7n^3$ vs. $T(n) \leq 100n^3$? The *shape* is n^3 either way. We want to ignore the constant.

The bound might fail for $n = 1, 2, 3$ but hold for all $n \geq 10$. We only need it for **sufficiently large** n .

Fix both $\Rightarrow$ **asymptotic notation**: O , Ω , Θ .

5 Big O (Upper Bound)

Big O (formal)

$T(n)$ is $O(g(n)) \iff \exists c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0$:

$$T(n) \leq c \cdot g(n)$$

For **every** input of size n (large enough), the algorithm takes **at most** $c \cdot g(n)$ steps.

Example: show $T(n) = 5n^2 + 3n$ is $O(n^2)$

Need c, n_0 such that $5n^2 + 3n \leq c \cdot n^2$ for all $n \geq n_0$.

For $n \geq 1$: $5n^2 + 3n \leq 5n^2 + 3n^2 = 8n^2$.

So $c = 8, n_0 = 1$. ✓

6 Big Ω (Lower Bound)

Big Ω (formal)

$T(n)$ is $\Omega(g(n)) \iff \exists c > 0, \exists n_0 > 0$ such that $\forall n \geq n_0$:

$$T(n) \geq c \cdot g(n)$$

For **some** input of size n (large enough), the algorithm takes **at least** $c \cdot g(n)$ steps.

O vs Ω in one line

O = **every** input, **at most**. Ω = **some** input, **at least**.

Example: show $T(n) = 5n^2 + 3n$ is $\Omega(n^2)$

Need c, n_0 such that $5n^2 + 3n \geq c \cdot n^2$ for all $n \geq n_0$.

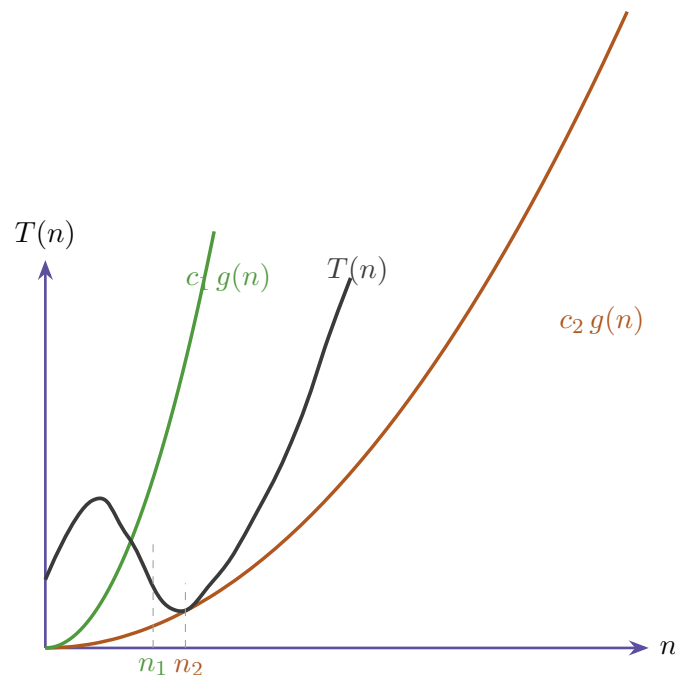
For $n \geq 1$: $5n^2 + 3n \geq 5n^2$.

So $c = 5, n_0 = 1$. ✓

7 Big Θ (Tight Bound)**Big Θ (formal)**

$T(n)$ is $\Theta(g(n)) \iff T(n)$ is $O(g(n))$ **and** $T(n)$ is $\Omega(g(n))$

Same $g(n)$ works as both **upper bound** and **lower bound**.



Past n_2 , $T(n)$ is trapped between the $c_2g(n)$ floor and the $c_1g(n)$ ceiling. That is Θ .

8 Worked Example: Bubble Sort is $\Theta(n^2)$ **BubbleSort(A)**

```
for i = 1 to A.length - 1:
  for j = A.length downto i + 1:
    if A[j] < A[j-1]: swap A[j] and A[j-1]
```

Bubble Sort: $T(n) = \Theta(n^2)$

$O(n^2)$: For **every** input, the nested loops do $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$ comparisons. So $T(n) \leq c_1 n^2$.

$\Omega(n^2)$: For **some** input (reverse-sorted: $[n, n-1, \dots, 1]$), every comparison triggers a swap, doing exactly $\frac{n(n-1)}{2}$ comparisons. So $T(n) \geq c_2 n^2$.

Same shape $\Rightarrow \Theta(n^2)$.

Practice: is $3n^2 + 5n \log n$ in $\Theta(n^2)$?

$O(n^2)$: $3n^2 + 5n \log n \leq 3n^2 + 5n^2 = 8n^2$ for $n \geq 1$. ✓

$\Omega(n^2)$: $3n^2 + 5n \log n \geq 3n^2$ for $n \geq 1$. ✓

Yes, $3n^2 + 5n \log n = \Theta(n^2)$.

Quick Reference: Lecture 1

$T(n)$	$\max\{t(x) \mid x \text{ of size } n\}$
$O(g(n))$	$T(n) \leq c g(n)$ eventually every input, at most
$\Omega(g(n))$	$T(n) \geq c g(n)$ eventually some input, at least
$\Theta(g(n))$	both O and Ω with same g (tight)
Bubble sort	$\Theta(n^2)$

Lecture 2: Priority Queues and Max-Heaps

9 ADT vs. Data Structure

Two different things

Abstract data type (ADT): specifies an **object** + **operations**. Says *what*, not *how*.

Data structure: **one specific implementation** of an ADT (memory layout + code for each operation).

Multiple data structures can implement the same ADT, with different costs.

10 The Priority Queue ADT

Priority Queue ADT

Object: a set S of elements, each with a comparable **key** (priority).

Operations:

- **Insert**(S, x): add element x into S .
- **Max**(S): return element with highest priority (no removal).
- **Extract_Max**(S): return **and remove** element with highest priority.

Application: OS job scheduling. Jobs arrive $\rightarrow$ **Insert**. Processor free $\rightarrow$ **Extract_Max** to run the highest-priority job.

11 Naive Implementations: Why We Need Something Better

Worst case	Insert	Extract_Max
Unordered linked list	$\Theta(1)$	$\Theta(n)$
Ordered linked list	$\Theta(n)$	$\Theta(1)$

Unordered: Insert at front is $\Theta(1)$, but Extract_Max scans the whole list $\Rightarrow \Theta(n)$.

Ordered: Max is at the end $\Rightarrow$ Extract_Max is $\Theta(1)$, but inserting in sorted position $\Rightarrow \Theta(n)$.

Goal

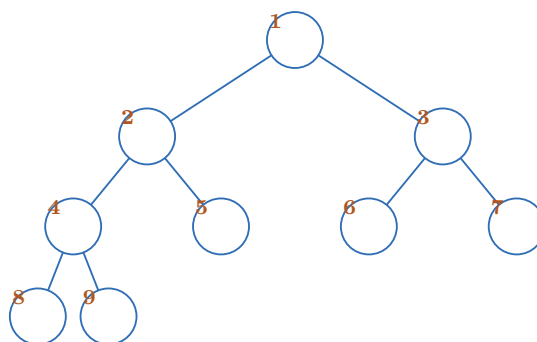
Both operations in $O(\log n)$. A **max-heap** achieves this.

12 Complete Binary Tree (CBT)

Complete Binary Tree

A binary tree where:

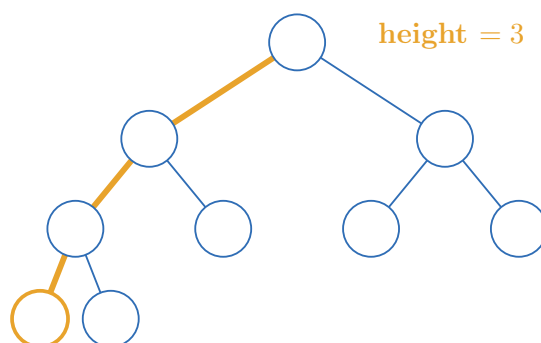
- every level except possibly the last is completely full,
- the last level is filled **from the left** with no gaps.



12.1 Height

Height of a tree

Number of **edges** on the longest root-to-leaf path.



Key fact

A CBT with n nodes has height $\lfloor \log_2 n \rfloor$.

Each full level doubles the node count (1, 2, 4, 8, ...), so $\log_2 n$ levels suffice for n nodes.

13 Max-Heap

Max-Heap = CBT + max-heap property

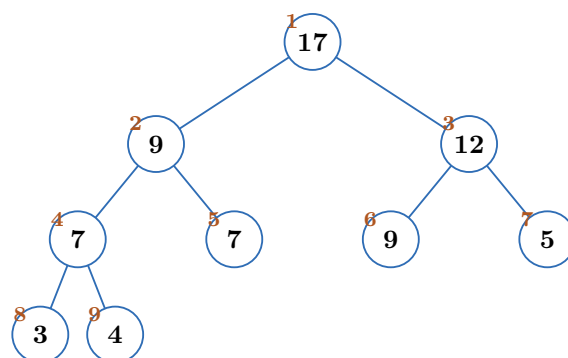
Elements stored in a CBT such that:

$$\text{priority}(\text{node}) \geq \text{priority}(\text{each child})$$

This is a **local** rule (parent vs. own children only). It chains upward, so the **global max is always at the root**.

Example

$S = \{3, 4, 5, 7, 7, 9, 9, 12, 17\}$. One valid max-heap:

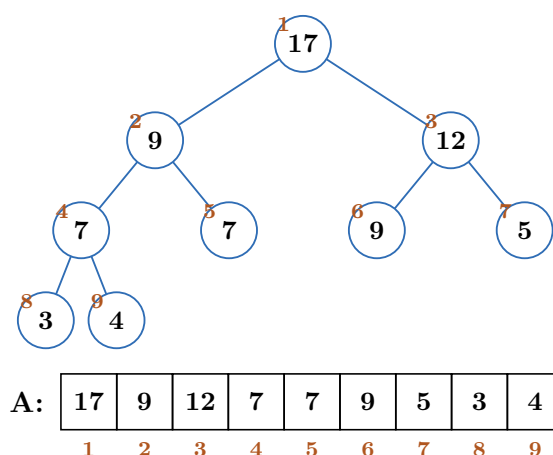


Check: $17 \geq 9, 12$. $9 \geq 7, 7$. $12 \geq 9, 5$. ✓

Swapping children of the root ($9 \leftrightarrow 12$) gives a different valid heap $\Rightarrow$ max-heaps are not unique. Only the root (global max) is guaranteed.

14 Array Representation

Store the tree **level by level, left to right** in array $A[1..n]$. No pointers needed.



Navigation formulas

For node at index i :

- Left child: $2i$ Right child: $2i + 1$ Parent: $\lfloor i/2 \rfloor$

$A.\text{heapsize}$ tracks the number of elements currently in the heap.

Verify: node at $i = 2$ holds 9

Left child: $A[2 \cdot 2] = A[4] = 7$ ✓ Right child: $A[2 \cdot 2 + 1] = A[5] = 7$ ✓

Parent: $A[\lfloor 2/2 \rfloor] = A[1] = 17$ ✓

Each level is $\sim 2\times$ the width of the one above, so going down doubles the index, going up halves it.

15 The Three Operations

Invariant for every operation

- 1) Maintain the **CBT shape**.
- 2) Maintain the **max-heap property**.

15.1 Max(A)

Max(A)

```
return A[1]
```

The root always holds the max. $\Rightarrow \Theta(1)$.

15.2 Insert(A, x): percolate up

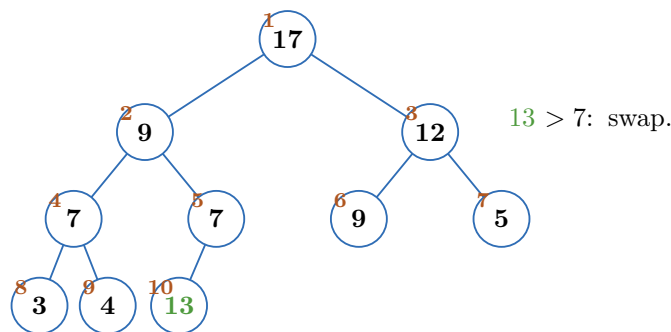
Insert(A, x)

1. $A.\text{heapsize}++$; $A[A.\text{heapsize}] = x$ *(place at next open leaf)*
2. while x is not root and $\text{priority}(x) > \text{priority}(\text{parent}(x))$:
 swap x with $\text{parent}(x)$ *(percolate up)*

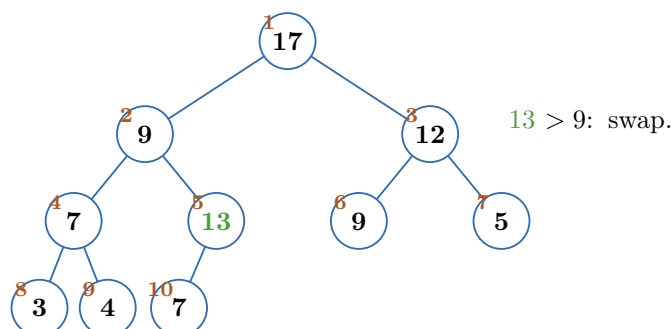
Worked example: Insert(A, 13)

Start: $A = [17, 9, 12, 7, 7, 9, 5, 3, 4]$. Insert 13 at index 10.

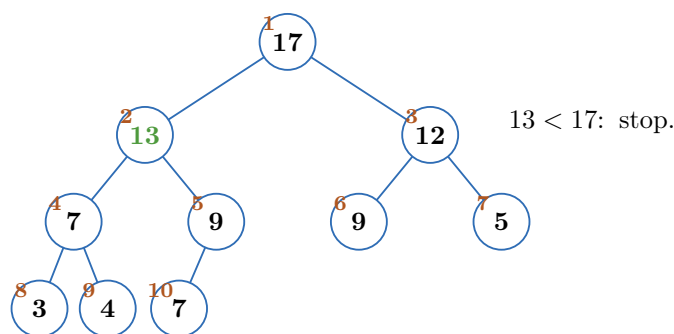
Step 1. Place 13 at $A[10]$. Property broken: $13 > 7$ (parent at $i = 5$).



Step 2a. Swap $13 \leftrightarrow 7$. Now 13 at $i = 5$. Compare: $13 > 9$ (parent at $i = 2$).



Step 2b. Swap $13 \leftrightarrow 9$. Now 13 at $i = 2$. Compare: $13 < 17$ (root). Stop.



Result: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, $\text{heapsize} = 10$.

15.3 Extract_Max(A): drip down

Extract_Max(A)

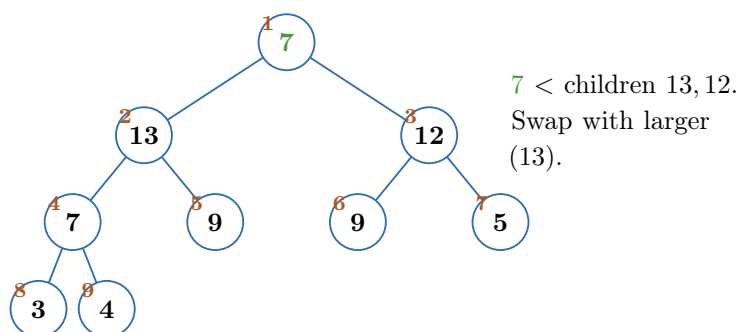
1. $\text{max} = A[1]$ *(save the root)*
2. $A[1] = A[A.\text{heapsize}]; A.\text{heapsize}--$ *(move last leaf to root)*
3. while x has a child with higher priority:
 swap x with its largest child *(drip down)*
4. return max

Important: always swap with the *larger* child (otherwise we create a new violation). Node at index i is a leaf when $i > \lfloor \text{heapsize}/2 \rfloor$.

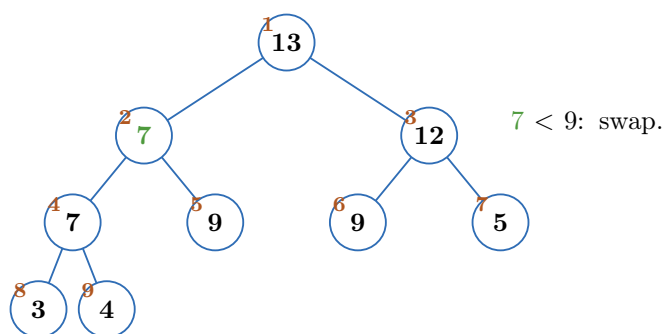
Worked example: Extract_Max(A)

Start: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, $\text{heapsize} = 10$.

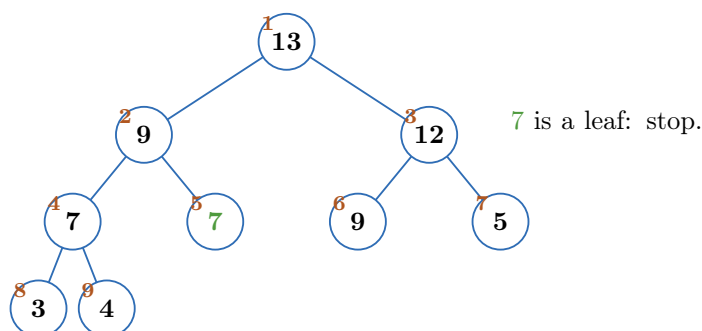
Steps 1 & 2. Save 17. Move $A[10] = 7$ to root. Shrink heapsize to 9.



Step 3a. Swap $7 \leftrightarrow 13$. Now 7 at $i = 2$. Children: 7, 9. Larger is $9 > 7$.



Step 3b. Swap $7 \leftrightarrow 9$. Now 7 at $i = 5$. Children at $i = 10 > \text{heapsize} = 9$, so 7 is a leaf. Stop.



Return 17. Heap: $[13, 9, 12, 7, 7, 9, 5, 3, 4]$.

16 Complexity Analysis

Each operation walks one root-to-leaf (or leaf-to-root) path. Path length $\leq \text{height} = \lfloor \log_2 n \rfloor$. Constant work per level.

Heap operation costs

Max (A)	$\Theta(1)$	read root
Insert (A, x)	$\Theta(\log n)$	percolate up $\leq \lfloor \log_2 n \rfloor$ levels
Extract_Max (A)	$\Theta(\log n)$	drip down $\leq \lfloor \log_2 n \rfloor$ levels

Insert = $\Theta(\log n)$:

- $O(\log n)$: At most $\lfloor \log_2 n \rfloor$ swaps (one per level). **every** input, **at most** $c_1 \log n$.
- $\Omega(\log n)$: Insert a value larger than the current root $\Rightarrow$ percolates all the way to the top. **some** input, **at least** $c_2 \log n$.

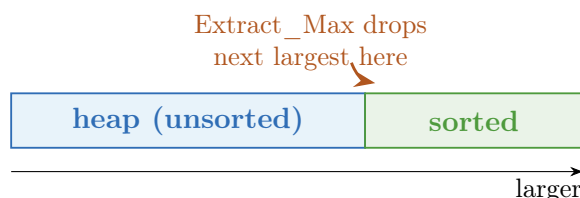
Same argument for **Extract_Max** (drip down with the smallest value $\Rightarrow$ sinks to a leaf).

17 HeapSort

HeapSort

1. Build a max-heap from A . ($\Theta(n)$ time)
2. Call **Extract_Max** n times. (each $\Theta(\log n)$)

Total: $n \times \Theta(\log n) = \Theta(n \log n)$



Each extraction moves the max to the front of the sorted region. The heap shrinks by one, the sorted region grows by one. After n extractions, the array is sorted in place.

18 Past Exam: k -Set ADT

★ Seen on a past exam (Midterm, Fall 2014, Q2)

The *rank* of an element = its position in sorted order (from 1). For fixed $k \in \mathbb{Z}^+$:

- **Object:** collection C of rationals (duplicates allowed).
- **Insert**(C, x): add x to C .
- **k -Select**(C): return element at rank k , or **nil** if $|C| < k$.

Q. Implement with data structures from class. **k -Select** must be $O(1)$.

Data structure: **max-heap** H_1 (stores the k smallest elements) + **min-heap** H_2 (stores the rest).

The k -th smallest element = the largest among the k smallest = root of H_1 .

k -Select(C):

```
if  $|H_1| < k$ : return nil
else: return  $H_1[1]$ 
```

$\Rightarrow O(1)$

Insert(C, x):

```
if  $|H_1| < k$  or  $x < H_1[1]$ : insert  $x$  into  $H_1$ 
else: insert  $x$  into  $H_2$ 
if  $|H_1| > k$ : insert Extract-Max( $H_1$ ) into  $H_2$ 
```

Constant number of heap ops, each $O(\log n) \Rightarrow O(\log n)$.

Correctness: H_1 always holds exactly the k smallest. New small values enter H_1 ; overflow gets pushed to H_2 . Large values go straight to H_2 .

Quick Reference: Lecture 2

Max-heap property	$\text{priority}(\text{node}) \geq \text{priority}(\text{children}); \text{max at root } A[1]$
Array navigation	$\text{left} = 2i, \text{ right} = 2i + 1, \text{ parent} = \lfloor i/2 \rfloor$
Height of CBT	$\lfloor \log_2 n \rfloor$
Max	read $A[1] \rightarrow \Theta(1)$
Insert	place at end, percolate up $\rightarrow \Theta(\log n)$
Extract_Max	move last to root, drip down $\rightarrow \Theta(\log n)$
HeapSort	build heap + n extractions $\rightarrow \Theta(n \log n)$

End of Lectures 1 & 2 notes.

Lecture 3: Min Binomial Heaps

19 Mergeable Priority Queue ADT

Last lecture: Priority Queue ADT (Insert, Max, Extract_Max). Now we flip to **min** (smallest key = highest priority) and add one more operation:

Mergeable Priority Queue ADT

Object: a set S of elements, each with a comparable **key**.

Operations:

- **Insert**(S, x): add element x to S .
- **Min**(S): return the element with the smallest key.
- **Extract_Min**(S): remove and return the element with the smallest key.
- **Union**(S_1, S_2): merge two priority queues into a single one.

Operation	Min Heap (array)	Min Binomial Heap
Insert	$O(\log n)$	$O(\log n)$
Min	$O(1)$	$O(\log n)$
Extract_Min	$O(\log n)$	$O(\log n)$
Union	$O(n)$	$O(\log n)$

A binary min heap stores elements in an array. To merge two heaps you basically dump them together and rebuild. That costs $O(n)$. Binomial heaps are specifically **designed** so Union runs in $O(\log n)$.

20 Binomial Trees

20.1 Recursive Definition of B_k

Binomial Tree B_k

Base case: B_0 = a single node.

Recursive case: B_k = take two copies of B_{k-1} . Make one root the **leftmost child** of the other root.

Equivalently: the root of B_k has children $B_{k-1}, B_{k-2}, \dots, B_1, B_0$ (left to right).

Breaking Down the Recursive Definition

This is a common confusion point, so let's be explicit.

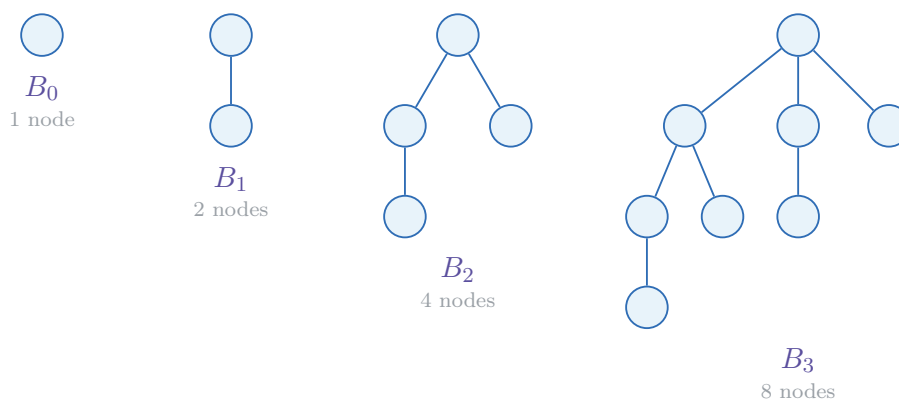
To build B_k :

1. Start with one copy of B_{k-1} . Call its root r .

2. Take a **second** copy of B_{k-1} .
3. Make the root of copy 2 the **new overall root**, and attach r 's entire tree as a subtree under it.

So $B_k = \text{two } B_{k-1}\text{'s glued together}$, one becoming a child of the other.

Building Up From B_0



20.2 Children Sequence of B_k

Children of the Root of B_k

The root of B_k has exactly k children. Reading **left to right**:

$$\underbrace{B_{k-1}}_{\text{leftmost (biggest)}}, B_{k-2}, \dots, B_1, \underbrace{B_0}_{\text{rightmost (single node)}}$$

The **leftmost** child is the **largest** subtree (B_{k-1}), and the **rightmost** child is the **smallest** (B_0). This ordering matters a lot for the memory representation later.

20.3 Properties of B_k

Key Properties of B_k

Height of B_k	$= k$
Number of nodes in B_k	$= 2^k$
Nodes at depth d in B_k	$= \binom{k}{d}$
Degree of root of B_k	$= k$

20.4 Why “Binomial”?

Why the Name “Binomial”

The number of nodes at depth d in B_k is $\binom{k}{d}$, the **binomial coefficient** (same “ n choose k ” from combinatorics).

Quick check for B_3 (8 nodes total):

Depth 0: $\binom{3}{0} = 1$ (the root)

Depth 1: $\binom{3}{1} = 3$

Depth 2: $\binom{3}{2} = 3$

Depth 3: $\binom{3}{3} = 1$

Total: $1 + 3 + 3 + 1 = 8$ ✓

21 Binomial Forest F_n

Binomial Forest F_n

A **Binomial Forest** F_n is a sequence of binomial trees $\langle B_{k_1}, B_{k_2}, \dots, B_{k_m} \rangle$ where:

- The k_i values are **strictly decreasing**: $k_1 > k_2 > \dots > k_m$
- The total number of nodes across all trees = n

21.1 Connection to Binary Representation

Binary Representation $\Leftrightarrow$ Which Trees Are Present

Write n in binary. B_i is present in F_n **if and only if** bit i (counting from the right, starting at 0) is **1**.

This is one of the most important ideas in this lecture. The entire structure of a binomial heap is dictated by the binary representation of n .

Example: F_7 and F_9

F_7 : $7 = \langle 111 \rangle_2$

All three bits are 1, so $F_7 = \langle B_2, B_1, B_0 \rangle$.

Sizes: $4 + 2 + 1 = 7$ ✓

F_9 : $9 = \langle 1001 \rangle_2$

Bits 0 and 3 are 1, so $F_9 = \langle B_3, B_0 \rangle$.

Sizes: $8 + 1 = 9$ ✓

21.2 What is $\alpha(n)$?

$\alpha(n)$ (popcount)

$\alpha(n)$ = the number of **1's** in the binary representation of n .

Other names: **popcount**, **Hamming weight**, **bit population count**.

Computing $\alpha(n)$

$$\alpha(7) = \alpha(\langle 111 \rangle_2) = 3$$

$$\alpha(9) = \alpha(\langle 1001 \rangle_2) = 2$$

$$\alpha(16) = \alpha(\langle 10000 \rangle_2) = 1$$

$$\alpha(15) = \alpha(\langle 1111 \rangle_2) = 4$$

21.3 Properties of F_n

Properties of F_n

Number of trees in $F_n = \alpha(n)$ (one per 1 bit)

Max number of trees $\leq \lfloor \log_2 n \rfloor + 1$

Number of edges in $F_n = n - \alpha(n)$

Verifying Edge Count for F_7

$F_7 = \langle B_2, B_1, B_0 \rangle$.

Edges in B_2 : 3, edges in B_1 : 1, edges in B_0 : 0. Total = 4.

Formula: $n - \alpha(n) = 7 - 3 = 4$ ✓

22 Min Binomial Heap: The Full Definition

Min Binomial Heap

A **Min Binomial Heap** of n elements is a Binomial Forest F_n such that:

1. **One element per node:** each node stores exactly one element.
2. **Min heap ordering:** in each tree, every node's key $\leq$ all keys in its subtree. (Root of each tree = smallest key in that tree.)

A Min Binomial Heap is a **collection of trees** (a forest). Each individual tree is a binomial tree satisfying the min heap property. Do not confuse this with a single binary heap.

23 Building a Binomial Heap: Worked Example

Build a Min Binomial Heap from $S = \{10, 13, 1, 3, 8, 18, 7\}$

$n = 7$, and $7 = \langle 111 \rangle_2$, so the final forest = $\langle B_2, B_1, B_0 \rangle$.

Insert 10: $n = 1 = \langle 1 \rangle_2 \Rightarrow \langle B_0 \rangle$

B_0 : 10

Insert 13: $n = 2 = \langle 10 \rangle_2 \Rightarrow \langle B_1 \rangle$

Two B_0 's $\rightarrow$ merge. Compare 10 vs 13: $10 \leq 13$, so 10 is root.

B_1 : 10 $\rightarrow$ 13

Insert 1: $n = 3 = \langle 11 \rangle_2 \Rightarrow \langle B_1, B_0 \rangle$

1 sits as the new B_0 , B_1 stays.

B_1 : 10 $\rightarrow$ 13 B_0 : 1

Insert 3: $n = 4 = \langle 100 \rangle_2 \Rightarrow \langle B_2 \rangle$

3 forms B_0 . Two B_0 's (1 and 3): $1 \leq 3 \Rightarrow B_1$ (root 1, child 3). Two B_1 's (root 1 vs root 10): $1 \leq 10 \Rightarrow B_2$ (root 1). **2 comparisons.**

B_2 : root 1, children: $B_1(10 \rightarrow 13)$ and $B_0(3)$

Insert 8: $n = 5 = \langle 101 \rangle_2 \Rightarrow \langle B_2, B_0 \rangle$

B_2 (root 1) B_0 : 8 **0 comparisons.**

Insert 18: $n = 6 = \langle 110 \rangle_2 \Rightarrow \langle B_2, B_1 \rangle$

Two B_0 's (8 and 18): $8 \leq 18 \Rightarrow B_1$ (root 8, child 18). **1 comparison.**

B_2 (root 1) B_1 : 8 $\rightarrow$ 18

Insert 7: $n = 7 = \langle 111 \rangle_2 \Rightarrow \langle B_2, B_1, B_0 \rangle$

7 is the new B_0 . No merges needed. **0 comparisons.**

Final heap: $\langle B_2(\text{root } 1), B_1(\text{root } 8), B_0(\text{root } 7) \rangle$

Total key comparisons: $0 + 1 + 0 + 2 + 0 + 1 + 0 = 4 = n - \alpha(n) = 7 - 3$

Important Facts

Each edge corresponds to exactly **one key comparison**
 Building from n elements takes $O(n)$ key comparisons total

24 Memory Representation

24.1 Three Pointers Per Node

Pointer Structure

Each node stores **three pointers** plus two data fields:

Field	Points To	Color
PARENT	Parent node	Red
key	Element's key	(data)
degree	Number of children	(data)
CHILD	Leftmost child	Green
SIBLING	Right sibling	Blue

We need the **PARENT** pointer for Decrease Key (covered later).

24.2 Visual vs. Memory Representation

When we draw a binomial heap, the lines are **edges** (parent to child).

In memory, the actual **pointers** are:

- **PARENT**: child $\rightarrow$ its parent (upward)
- **LEFT CHILD**: parent $\rightarrow$ its **leftmost** child only
- **RIGHT SIBLING**: one child $\rightarrow$ the next sibling (across)

The visual edges are NOT pointers. Each visual edge is implemented through a combination of LEFT CHILD and RIGHT SIBLING pointers.

24.3 HEAD Pointer and Root List

The roots of the B_k trees are connected via **RIGHT SIBLING** pointers, forming a **linked list of roots**. A **HEAD** pointer points to the root of the **smallest** tree (smallest k).

Memory Layout Example

Heap: $\langle B_3(\text{root } 8), B_2(\text{root } 1), B_0(\text{root } 7) \rangle$

Root linked list:

HEAD $\rightarrow$ 7 $\xrightarrow{\text{sib}}$ 1 $\xrightarrow{\text{sib}}$ 8 $\rightarrow$ NIL

HEAD points to 7 (B_0 , smallest tree). Then 7's sibling pointer goes to 1 (B_2), then to 8 (B_3).

25 Two Key Lemmas

Before the operations, we need two building blocks. These lemmas are **the reason** all operations work efficiently.

25.1 Lemma 1: Merging Two B_k Trees

Lemma 1

We can merge **two** min heap ordered B_k trees into a **single** min heap ordered B_{k+1} tree using just **one** key comparison.

Why Lemma 1 Matters

This is the **carry operation**. Just like adding two 1 bits produces a carry in binary addition, merging two B_k 's produces one B_{k+1} . This is the core mechanism behind Union, Insert, and Extract_Min.

Proof. Given two B_k trees with roots x and y :

1. Compare $x.\text{key}$ and $y.\text{key}$ (one comparison).
2. Smaller root stays on top. Make the other tree the new leftmost subtree.

If $x \leq y$: x is root of B_{k+1} , with children $B_k(y)$, B_{k-1} , $\dots$, B_0 .

Min heap ordering preserved (smaller root on top). Structure is B_{k+1} by definition. $\square$

25.2 Lemma 2: Deleting the Root of B_k

Lemma 2

Deleting **the root** of a min heap ordered B_k gives a valid **min binomial heap** of $2^k - 1$ elements.

Why Lemma 2 Matters

This is the key insight behind Extract_Min. When we extract the minimum (a root of some B_k tree), the remaining children form their own binomial heap. We then Union that new heap back with the rest.

Proof. Root of B_k has children $B_{k-1}, B_{k-2}, \dots, B_1, B_0$.

Delete the root $\Rightarrow$ left with these k subtrees. Each is already min heap ordered (removing the root does not affect subtrees). The sequence $\langle B_{k-1}, \dots, B_1, B_0 \rangle$ is a valid binomial forest (strictly decreasing k values), so it is a binomial heap of size $2^{k-1} + \dots + 1 = 2^k - 1$. $\square$

26 Operations

The Big Picture

Everything reduces to Union.

- Insert = Union with a tiny heap (single B_0)
- Extract_Min = remove a root (Lemma 2 gives a new heap) + Union
- Union itself = binary addition using Lemma 1 as the carry

Lemma 1 ($O(1)$ merge) and Lemma 2 (delete root $\Rightarrow$ valid heap) are the two pillars.

26.1 Union(T, Q): Merging Two Binomial Heaps

Union(T, Q)

Input: two min binomial heaps T and Q .

Output: a single min binomial heap S containing all elements.

Algorithm: Binary addition!

Each column (bit position i) can have 0, 1, 2, or 3 copies of B_i (from T , from Q , plus possibly a carry from position $i - 1$). Process right to left:

- **0 trees:** nothing in S , no carry
- **1 tree:** put it in S , no carry
- **2 trees:** merge via Lemma 1 into B_{i+1} , carry it. Nothing in S at position i .
- **3 trees:** put one in S , merge the other two into a carry

Full Worked Example: Union(T, Q)

T has size $3 = \langle 11 \rangle_2$, so $T = \langle B_1, B_0 \rangle$.

Q has size $7 = \langle 111 \rangle_2$, so $Q = \langle B_2, B_1, B_0 \rangle$.

Result: $3 + 7 = 10 = \langle 1010 \rangle_2$, so $S = \langle B_3, B_1 \rangle$.

Concrete values:

- T : B_1 (root 2, child 4). B_0 (root 3).
- Q : B_2 (root 5, children: 7→9 and 10). B_1 (root 1, child 8). B_0 (root 6).

Position 0 (B_0 column):

T has $B_0(3)$, Q has $B_0(6)$. Two trees $\Rightarrow$ merge.

Compare 3 vs 6: $3 \leq 6 \Rightarrow B_1$ (root 3, child 6). This is the **carry**.

S at position 0: empty.

Position 1 (B_1 column):

T has B_1 (root 2), Q has B_1 (root 1), plus carry B_1 (root 3). **Three trees!**

Keep one in S : the carry $B_1(3 \rightarrow 6)$.

Merge the other two: compare 1 vs 2: $1 \leq 2 \Rightarrow B_2$ (root 1, with subtrees from both). New carry.

S at position 1: B_1 (root 3, child 6).

Position 2 (B_2 column):

Q has B_2 (root 5), plus carry B_2 (root 1). Two trees $\Rightarrow$ merge.

Compare 1 vs 5: $1 \leq 5 \Rightarrow B_3$ (root 1). Carry to position 3.

S at position 2: empty.

Position 3 (B_3 column):

Just the carry B_3 (root 1). One tree $\Rightarrow$ done.

S at position 3: B_3 (root 1).

Result: $S = \langle B_3(\text{root } 1), B_1(\text{root } 3) \rangle$ $10 = \langle 1010 \rangle_2$ ✓

New edges: 3 **Key comparisons:** 3

Union Complexity

If $|T| \leq n$ and $|Q| \leq n$, each heap has $\leq \lfloor \log_2 n \rfloor + 1$ trees.
 $\Rightarrow$ Union takes $O(\log n)$ comparisons (one per column of the binary addition).

26.2 $\text{Insert}(T, x)$: Adding a Single Element

$\text{Insert}(T, x)$

1. Create a new heap Q containing just x (a single B_0).
2. Return $\text{Union}(T, Q)$.

Insert = Adding 1 in Binary

If the last bit is 0 (no B_0 exists): place the new B_0 . Done in $O(1)$.
 If the last bit is 1: merge into B_1 (carry). If B_1 already exists, merge again into B_2 . This cascading carry can go up to $O(\log n)$ positions, but on average much less.

Insert Example

Heap $T = \langle B_2, B_0 \rangle$ (5 elements, binary 101). Insert element x .

1. x becomes B_0 . Now two B_0 's.
2. Merge them $\Rightarrow$ one B_1 (carry). No B_1 exists, so B_1 stays.
3. Result: $\langle B_2, B_1 \rangle$ (6 elements, binary 110). $\checkmark$

26.3 $\text{Min}(T)$: Finding the Minimum

$\text{Min}(T)$

The minimum element is the root of one of the trees (by min heap property).
 Walk through the root list (linked list via HEAD pointer). Compare each root's key. Return the smallest.
 At most $\lfloor \log_2 n \rfloor + 1$ roots $\Rightarrow O(\log n)$.

26.4 $\text{Extract_Min}(T)$: Remove the Minimum

$\text{Extract_Min}(T)$

1. **Find the min:** walk the root list to find the tree B_k whose root has the smallest key. $O(\log n)$
2. **Remove that tree** from the heap T . $O(1)$
3. **Delete the root** of B_k . By **Lemma 2**, the children form a new binomial heap $Q = \langle B_{k-1}, \dots, B_0 \rangle$. $O(\log n)$
4. **Union**(T, Q). $O(\log n)$

Total: $O(\log n)$.

Extract_Min Example

Heap: $\langle B_3(\text{root } 1), B_1(\text{root } 3) \rangle$ (10 elements)

1. $\text{Min} = 1$ (root of B_3).
2. Remove B_3 from heap. Remaining: $\langle B_1(\text{root } 3) \rangle$.
3. Delete root 1 from B_3 . Children: B_2, B_1, B_0 . New heap $Q = \langle B_2, B_1, B_0 \rangle$ (7 elements).
4. $\text{Union}(\langle B_1 \rangle, \langle B_2, B_1, B_0 \rangle)$:
Position 0: one $B_0 \Rightarrow$ stays.
Position 1: two B_1 's $\Rightarrow$ merge into B_2 (carry).
Position 2: two B_2 's $\Rightarrow$ merge into B_3 (carry).
Position 3: one $B_3 \Rightarrow$ stays.
Result: $\langle B_3, B_0 \rangle$ (9 elements, $9 = \langle 1001 \rangle_2$) $\checkmark$

Return 1.

27 Complexity Summary

Binomial Heap Operations		
Operation	Worst Case	Key Insight
$\text{Union}(T, Q)$	$O(\log n)$	binary addition
$\text{Insert}(T, x)$	$O(\log n)$	Union with single B_0
$\text{Min}(T)$	$O(\log n)$	scan root list
$\text{Extract_Min}(T)$	$O(\log n)$	Lemma 2 + Union

28 Practice Problems

Problem 1: Build a Binomial Heap

Build a min binomial heap by inserting: 5, 12, 3, 9, 2, 15, 6, 1.

- (a) How many trees does the final heap have?
- (b) What is the root of each tree?
- (c) How many total key comparisons?

$n = 8 = \langle 1000 \rangle_2 \Rightarrow F_8 = \langle B_3 \rangle$ (one tree).

Step by step:

- Insert 5: $\langle B_0(5) \rangle$. 0 comps
- Insert 12: merge B_0 's: $5 \leq 12$. $\langle B_1(5 \rightarrow 12) \rangle$. 1 comp
- Insert 3: $\langle B_1(5), B_0(3) \rangle$. 0 comps
- Insert 9: merge B_0 's: $3 \leq 9$. Two B_1 's: $3 \leq 5$. $\langle B_2(\text{root } 3) \rangle$. 2 comps
- Insert 2: $\langle B_2(3), B_0(2) \rangle$. 0 comps
- Insert 15: merge B_0 's: $2 \leq 15$. $\langle B_2(3), B_1(2 \rightarrow 15) \rangle$. 1 comp
- Insert 6: $\langle B_2(3), B_1(2), B_0(6) \rangle$. 0 comps
- Insert 1: merge B_0 's: $1 \leq 6$. Two B_1 's: $1 \leq 2$. Two B_2 's: $1 \leq 3$. $\langle B_3(\text{root } 1) \rangle$. 3 comps

(a) 1 tree. (b) Root is 1. (c) $1 + 0 + 2 + 0 + 1 + 0 + 3 = 7 = n - \alpha(n) = 8 - 1$

Problem 2: Extract_Min

Given: $\langle B_2(\text{root } 3, \text{children: } 7 \rightarrow 9 \text{ and } 5), B_0(\text{root } 10) \rangle$. Perform Extract_Min.

1. Min = 3 (root of B_2). Extract it.
2. Remove B_2 . Remaining: $\langle B_0(10) \rangle$.
3. Root 3 had children: $B_1(\text{root } 7, \text{child } 9)$ and $B_0(\text{root } 5)$. New heap $Q = \langle B_1(7 \rightarrow 9), B_0(5) \rangle$.
4. Union($\langle B_0(10) \rangle, \langle B_1(7), B_0(5) \rangle$):
 Position 0: two B_0 's (10, 5). $5 \leq 10 \Rightarrow B_1(\text{root } 5, \text{child } 10)$. Carry.
 Position 1: $B_1(\text{root } 7) + \text{carry } B_1(\text{root } 5)$. $5 \leq 7 \Rightarrow B_2(\text{root } 5)$. Carry.
 Position 2: just carry. Done.
 Result: $\langle B_2(\text{root } 5) \rangle$ (4 elements, $4 = \langle 100 \rangle_2$) ✓

Return 3.

Problem 3: Properties Quick Check

For B_5 :

Nodes?	$2^5 = 32$
Height?	5
Degree of root?	5
Nodes at depth 2?	$\binom{5}{2} = 10$
Nodes at depth 5?	$\binom{5}{5} = 1$

For F_{13} ($13 = \langle 1101 \rangle_2$):

Which trees?	$\langle B_3, B_2, B_0 \rangle$
How many trees?	$\alpha(13) = 3$
How many edges?	$13 - 3 = 10$

29 Exam Relevance

Based on past CSC263 exams (2012 through 2015), no specific binomial heap questions appeared. However, heap questions show up frequently. The k Set ADT problem from Fall 2014 (see Lecture 2 notes) used max heaps and min heaps together.

- “Draw the binomial heap after inserting $x_1, x_2, \dots$ ” $\Rightarrow$ binary addition method
- “Show the result of Extract_Min” $\Rightarrow$ Lemma 2 + Union
- “How many key comparisons?” $\Rightarrow$ count new edges = count merges
- “Why $O(\log n)$?” $\Rightarrow$ at most $\lfloor \log_2 n \rfloor + 1$ trees, binary addition has that many columns

Quick Reference: Lecture 3

B_k	two B_{k-1} ’s merged; has 2^k nodes, height k , $\binom{k}{d}$ at depth d
F_n	forest of B_k ’s matching 1 bits of n ; has $\alpha(n)$ trees, $n - \alpha(n)$ edges
Min Binomial Heap	F_n + one element per node + min heap ordering
Memory	3 pointers: PARENT, LEFT CHILD, RIGHT SIBLING; HEAD $\rightarrow$ root list
Lemma 1	merge two $B_k \rightarrow B_{k+1}$ with 1 comparison (the “carry”)
Lemma 2	delete root of $B_k \rightarrow$ valid heap of $2^k - 1$ elements
Union	binary addition $\rightarrow O(\log n)$
Insert	Union with $B_0 \rightarrow O(\log n)$
Min	scan root list $\rightarrow O(\log n)$
Extract_Min	Lemma 2 + Union $\rightarrow O(\log n)$